

Agentic Engineering

Your harness. Your skills. Your judgment.

Prompt → **Implement** → **Verify** → **Review** → **Improve**

You own the final review, approval and commit.

01 Choose your model

Workshop picks: Fable + Astra

Compare them on the same task and acceptance cases.

Judge correctness, review time, repairs and total cost.

Re-evaluate with each new model generation.

02 Set up the harness

The repo is the prompt.

AGENTS.md: shared rules and linked style guides. Point each agent to that source.

Wire ESLint, Prettier, hooks, build and tests. Add Angular MCP; other tools as needed.

Set permissions. Keep secrets outside the checkout.

Prove it: inspect loaded instructions, run a tool, then verify a small feature.

03 Use skills. Build your own.

.agents/skills/<name>/SKILL.md

Start with **create-a-skill** and one useful task skill.

Define triggers, exclusions, steps and checks. Keep it focused; load references when needed.

Test a matching task and one outside its scope. Review, version and share improvements.

Always-on rule → **instructions**

Repeatable judgment → **skill**

Deterministic work → **script or schematic**

04 Give a checkable prompt

Observation: what do you see?

Goal: what outcome do you want?

Constraints: what must stay unchanged?

Scope: which files or feature?

Check: what observable result proves it works?

Unclear? Ask questions one at a time before coding.

Bring an idea to **brainstorming**; a plan to **grill-me**.

One task per session. Use **handover** for continuity.

05 Verify, then review

pnpm workshop:check

Build + lint + formatting check + unit tests.

Also check requested behavior in tests or Chrome. Green checks alone do not prove a feature works.

Review twice: **requirements** first, then **conventions**.

Use an independent reviewer. Verify every finding.

Keep diffs small. **You stage and commit if you want to stay in control.**

06 Match workflow to risk

Small task: implement, verify, review.

Add a **spec** to clarify outcomes, a **plan** for sequencing and **phases** for larger diffs.

Review each boundary in a live session.

Autonomy: checkable goal, feedback, scope, permissions, runtime and spend caps.

Stop at success, a limit or an evidenced blocker.

Parallel tasks: independent work, separate worktrees.

Every error must improve the instructions

Reproduce → **Fix** → **Save the lesson** → **Verify again**

Record the cause and a concrete prevention rule in **AGENTS.md**, the relevant style guide or skill. Add an automated check where possible.

Check discovery and conflicts. Test in a fresh session. Remove duplication and retest older rules as the harness improves.

Refactor in small, verified steps

Analyse → **Declutter** → **Update** → **Modernize** → **Type** → **Refactor** → **Review**

Before refactoring, add an e2e safety net. Preserve meaning. Prefer official migrations. Keep TypeScript strict, use signals where appropriate and avoid needless abstractions.

This week: improve one rule · build one skill · wire one feedback loop